UIR · Integrated AI/ML Project

CV Screening Multi-Agent System

Intelligent candidate pre-screening
with specialist agents, Deep Learning & Human-in-the-Loop

2 Specialist Agents

PyTorch Model

Human Approval Gate

JSON Audit Trail

Zouhair Eouichak · Malak Bartal · Aymane El Youssi

Academic Year 2025 – 2026

Problem & Motivation

⚠ The Problem

Recruiters receive hundreds of CVs per position.

Manual screening is:

- Slow and time-consuming
- Subjective and inconsistent
- Hard to justify or audit

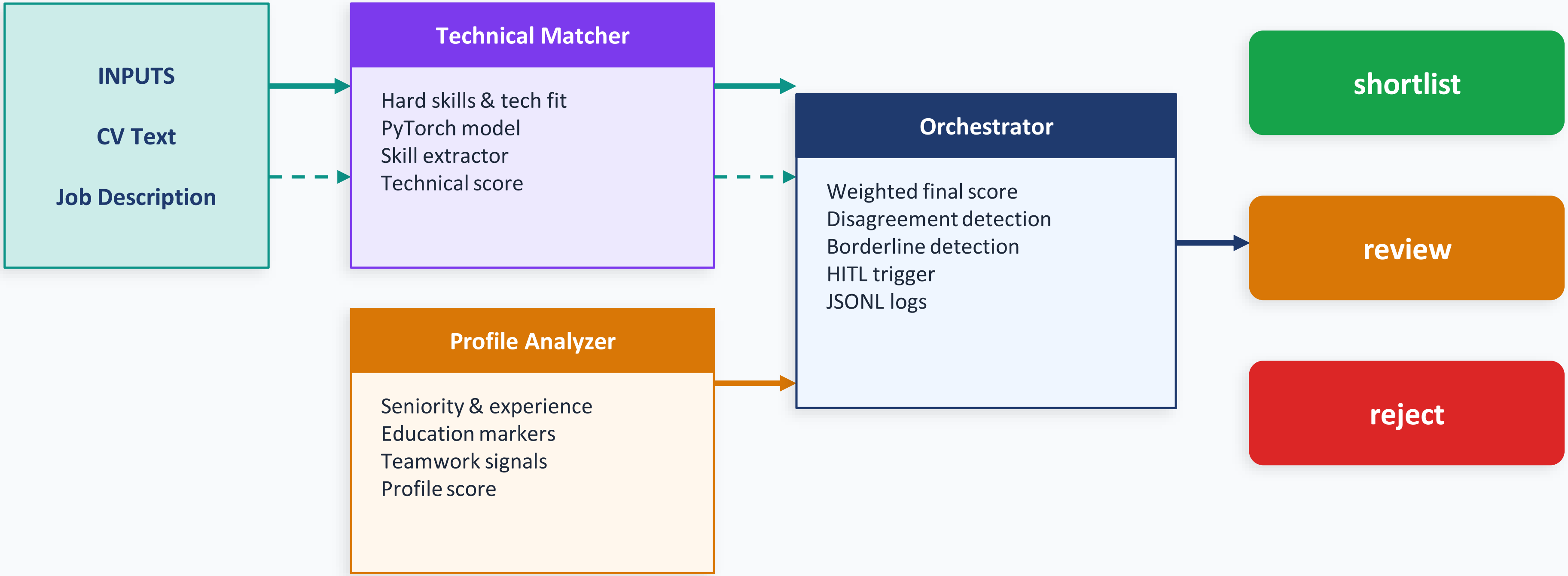
Use case: recruiting a Junior Data Analyst

✓ Our Solution

- Separate technical and profile analysis into specialist agents
- Use a trained PyTorch model for scoring
- Provide clear, explainable outputs
- Escalate uncertain cases to human review
- Log all decisions for traceability

02

Multi-Agent Architecture



⚠ Human-in-the-Loop triggered if: borderline score | agent disagreement | tool failure

03

Tools & Deep Learning Model

Tool 1 — DL Model Tool

Input: CV + Job description text

Output: Fit label (High/Medium/Low) + score

Framework: PyTorch 2.x — CVFitNet

File: models/cv_fit_model.pt

Tool 2 — Skill Extractor

Input: CV + Job description text

Output: Matched skills, missing skills, coverage ratio

Method: NLP keyword extraction

Usage: Technical Matcher + explainability

PyTorch Architecture — CVFitNet

End-to-End Workflow

1. **Load Inputs**
   CV (text/PDF/JSON) + Job description

2. **Technical Analysis**
   Technical Matcher PyTorch + Skill Extractor

3. **Profile Analysis**
   Profile Analyzer Seniority + Experience

4. **Orchestrator**
   Weighted score Label + Recommendation

5. **HITL Gate**
   Borderline / Conflict → Human review

6. **Output + Logs**
   JSON / CSV JSONL Audit Trail

Every step is logged to a structured JSONL audit file for full traceability and reproducibility.

Human-in-the-Loop (HITL)

HITL Triggers

- Borderline score (0.45 – 0.55)
- Strong disagreement between agents
- Tool or agent failure / fallback

HR Reviewer Actions

- Shortlist
- Needs Review
- Reject

Streamlit Mode

Blocking UI — explicit reviewer decision before the final recommendation is shown

CLI Mode

--require-human-approval marks cases as pending-human-approval

Evaluation & Metrics

100%
Model Accuracy
60 samples

100%
Macro F1 Score
Low/Med/High

83.3%
Pipeline Accuracy
6 eval cases

16/16
Tests Passed
pytest suite

Confusion Matrix (model)

	Pred: Low	Pred: Med	Pred: High
True: Low	24	0	0
True: Med	0	23	0
True: High	0	0	13

Pipeline Evaluation — 6 cases

Case	Expected	Predicted	Score
eval_001	High	High ✓	0.982
eval_002	Medium	Medium ✓	0.607
eval_003	High	Medium ✗	0.627
eval_004	Medium	Medium ✓	0.479
eval_005	Low	Low ✓	0.252
eval_006	Low	Low ✓	0.371

Streamlit Frontend — Dashboard Demo

Mode 1 — Single Screening

- 1 CV vs 1 job description
- 5 result tabs: Intake · Agent Analysis · Decision Room · Evidence · Audit Trail
- Blocking HITL gate for borderline cases
- Export JSON / CSV
- Downloadable Defense Pack

Mode 2 — Batch Screening

- Multiple CVs vs 1 job
- Ranked leaderboard by score
- Interactive bar chart
- Sources: Sample / JSON / PDFs
- Export JSON + CSV · Session history

```
$ streamlit run src/app_frontend.py
```

Runtimes & Frameworks

PRIMARY

CrewAI

Main runtime

Autonomous agents with LLM

Requires Ollama backend

`python -m src.main --runtime auto`

FALLBACK

Deterministic

Resilient fallback

Pure Python orchestrator

Works without any LLM

`Active when CrewAI unavailable`

BONUS

LangGraph

Stretch goal

Minimal scaffold included

Graph-based orchestration base

`Future extension`

Ollama LLM Backend — llama3.2 @ localhost:11434 · Enriches output with professional HR recommendation text · Does not affect agent scores

Logging & Traceability

Every agent action is logged to a structured JSONL file:

`timestamp`	ISO UTC of the event
`agent_name`	technical_matcher | profile_analyzer | orchestrator
`action`	Name of the action executed
`tool_used`	model_tool | skill_extractor | —
`input_summary`	Summary of the input processed
`output_summary`	Summary of the output produced
`status`	success | failure | pending
`error`	Error message if failure

```
logs/run_20260524T120000Z.jsonl   ·   logs/pipeline_evaluation.json
```

The Audit Trail tab in Streamlit displays the last 30 events in real time.

Challenges & Solutions

1

✗ **Challenge: Ollama latency during live demo**

✓ Solution: Server readiness checks + reduced max tokens + run_project.ps1 launcher

2

✗ **Challenge: CLI input() blocking Streamlit**

✓ Solution: Strict HITL mode with explicit reviewer UI decision (Shortlist / Reject / Needs Review)

3

✗ **Challenge: Port conflicts at startup**

✓ Solution: PowerShell launcher with configurable port and stale-process cleanup

Limitations: keyword-based skill extraction · Local hardware dependency for Ollama · Synthetic training dataset

Conclusion & Next Steps

Achievements

- ✓ 2 specialist agents + 1 orchestrator
- ✓ PyTorch model — 100% accuracy
- ✓ 2 tools integrated in agent workflow
- ✓ Blocking HITL (CLI + Streamlit)
- ✓ 16/16 tests passed
- ✓ Structured JSONL logs
- ✓ Batch screening + leaderboard
- ✓ CrewAI primary + deterministic fallback

Future Improvements

- → Semantic embedding-based skill matching
- → Bias & fairness analysis metrics
- → Recruiter feedback loop
- → Real-time analytics dashboard
- → Multilingual CV/job support

Thank you — Questions?